The Repository Context Layer

The Missing Layer of Agentic Software Development

Pavel Kerbel
Independent Researcher

Abstract. Repositories version source code, tests, documentation, and configuration. None of these gives an agent a standardized, evolving operating context. The decisions, the rejected alternatives, the constraint discovered at two in the morning: that knowledge lives in chat scrollback and in people's heads, and every agent session starts without it. This note proposes the *Repository Context Layer*, a version-controlled, human-readable store that agents consult before planning and update after executing, under human review. Git stores what changed; Repository Context stores what the project knows. Everything else is decoration.

Index Terms. repository context layer, context.md, agentic software development, institutional knowledge, decision records, self-improving repositories, consult-before-plan.

I. AGENTS WITH AMNESIA

An agent session begins with a strong model and an empty head. It can read every file in the repository and still not know why any of them look the way they do. Code records outcomes; the reasons were deleted somewhere between the whiteboard and the merge.

So the agent re-litigates settled arguments. It proposes the ORM you removed in March. It reintroduces the abstraction that failed in production. None of this is a model problem. The knowledge it needs exists, just nowhere a model can reach, because the most expensive knowledge in a repository is negative: what was tried and rejected. No standard artifact is versioned with the code, required reading for agents, and continuously updated through execution. The bill arrives as repeated mistakes, architectural drift, and a permanent tax of re-explaining the same project to the same model.

II. WHY EXISTING ARTIFACTS ARE NOT ENOUGH

The obvious rebuttals deserve answers. READMEs describe how to use a project, not how to change it, and nobody updates them when a decision is reversed. ADRs are the closest ancestor, but they are write-once essays for humans; no agent is required to read them and none is expected to append to them. RAG retrieves by similarity, which is precisely wrong for constraints: a rule matters most when nothing in the prompt resembles it. IDE memories are private to one tool and one machine, invisible to review. Agent instruction files carry orders from the human downward but have no defined way to absorb what the agent itself learns. Each solves a slice. None is versioned with the code, consulted by contract, and writable by the agent under human review. That combination is the missing layer.

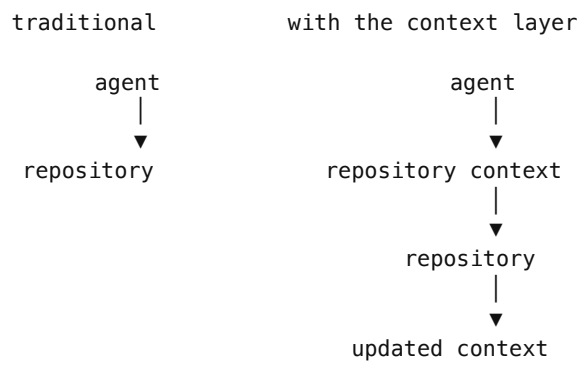
III. THE REPOSITORY CONTEXT LAYER

> *A Repository Context Layer is a version-controlled, human-readable context store that lives alongside the codebase and serves as the authoritative operating context for AI agents working in it.*
>
> *Git stores what changed.*
> *Repository Context stores what the project knows.*

Call it context, not memory. Memory is personal, fuzzy, optional; it evaporates with the session that formed it. Context is the deterministic control plane for the model's runtime behavior: on disk, versioned with the code, at a known address. It stops being a pile of notes the moment the agent is *required* to consult it and *expected* to maintain it.

IV. THE LIFECYCLE

consult → execute → update → commit



Before planning, the agent reads the context layer; a plan made without priors is a guess with good formatting. After executing comes the step almost every system skips: the agent writes back what the work taught it, whether that is a package that breaks the ARM64 build or a proxy timeout nothing documents.

The commit carries both the code and the sharpened context. No external store can offer this. Context evolves with the repository because it travels with it, branching when the code branches, merging when it merges, rolling back when it rolls back.

V. DESIGN PRINCIPLES

One choice does most of the work: the layer lives in Git. Git already solved provenance, review, branching, merging, distributed synchronization, and audit. Rather than inventing a new persistence layer for agent knowledge, the Repository Context Layer reuses the machinery engineers already trust, and inherits its guarantees for free. If you cannot `git log` your agent's beliefs, you cannot debug them. The remaining principles follow from that choice.

Human-readable. Plain Markdown. Readable in review, editable in any editor.

Reviewable. When an agent changes its own operating rules, the change appears in the diff next to the code that motivated it, and a human approves both at once. Self-modification with a human veto: an agent that edits its context in the open is safer than one that remembers in the dark.

Branch-aware. Context follows the branching model for free; no second source of truth to reconcile.

Tool-independent. No SDK, no server, no vendor. Any agent that can read a file participates, and any human with a text editor is a first-class writer.

Incrementally evolving. Start with three sections and one honest sentence in each; the layer grows when the project learns, not when a template demands it.

Deterministic discovery. The agent must find the context without searching for it. A fixed path is an API.

VI. MINIMAL SPECIFICATION

The proposed default implementation is the dullest thing possible: a single Markdown file, `context.md`. Discovery order is `.repo/context.md`, then `context.md` at the repository root; first hit wins. Three sections are required:

Intent: what this project is, and the design philosophy everything else must serve. *Constraints*: the non-negotiable rules, each with its reason. A rule without a reason is a superstition; the agent will comply but can never generalize. Record the rejection, not just the rule. *Evolved Context*: an append-only ledger of what agents and humans learned while working here. Entries that prove out graduate into Constraints, and that reviewed promotion is the self-improvement loop made tangible.

```
# Repository Context

## Intent
Local-first CLI. Files are the source of
truth; SQLite is a rebuildable index.

## Constraints
- No ORM. Rejected 2026-03: query opacity
  broke the offline repair path.

## Evolved Context
- [2026-06-29] pkg X >=3.0 breaks ARM64
  builds. Pin to 2.3.x until fixed.
```

Note what is standardized here: discovery and lifecycle, not content. Repositories will evolve different context structures, but every agent should know where to look and how the context changes. Everything beyond the three headers (decision logs, pattern catalogs, per-directory context) is convention layered on top. Do not over-specify. Under-specified and adopted beats complete and ignored.

VII. A REFERENCE IMPLEMENTATION

Metatron is one implementation of the layer: decisions and project facts kept as git-backed Markdown, served to agents at consult time, feedback routed into the evolved-context ledger. An optional index accelerates retrieval; the files remain the truth. It is an implementation, not the architecture. Any tool, or none at all, can implement the layer, and the abstraction should outlive every product built on it, including this one.

VIII. VERSION THE CONTEXT

Repositories are about to change population: soon most readers and writers of a codebase will not be human. Model weights are frozen between releases; repositories do not have to be. With a context layer, every session ends with sharper priors than the last, compounding at merge speed rather than training speed. The repository gets smarter, not the model, and self-improvement becomes a merge instead of a fine-tune. The unit of learning is a reviewed line in `context.md`.

Repositories learned to version code decades ago, then tests, then documentation, then infrastructure. The next step is to version context.